

Sentinelleai — Audit Report

IMagnetaOFTStandardFactory.sol · f196db8f-aa7b-4042-9212-089f8fe3900b · 2026-05-22
11 h 40 min 42 s

FAIL 28 / 100 8 consolidated findings

CreateTokenDispatcherV3 presents a CRITICAL vulnerability (SC05/OWASP) in its `\_lzReceive` handler: the function processes arbitrary cross-chain payloads without validating `\_origin.sender` against the registered peer for `\_origin.srcEid`, directly mirroring the Kelp DAO DVN-spoofing attack pattern (\$292M, 2026-04-19). Any attacker on any supported chain can forge token-creation messages with arbitrary `creator` addresses. A HIGH-severity reentrancy ordering issue (SC08) exists in `createTokenAtomic` where the `feeVault` external call precedes local token creation and cross-chain sends; while `nonReentrant` partially mitigates same-contract reentry, a malicious or compromised `feeVault` can still exploit the ordering. The RELAY-1 finding (SC05) regarding replay protection is partially mitigated by LayerZero's built-in GUID deduplication but the missing peer check makes it moot until the primary issue is fixed. FACT-1 array-length mismatch is already handled in code and is a false positive. FACT-2 zero-address concern applies to the factory interfaces (out of scope here) and is LOW risk in this contract. Formalist-generated invariants for upgradeability, timelock, oracle staleness, and ERC4626 are not applicable to this contract and are rejected. The contract must not be deployed until the trusted-sender check is added to `\_lzReceive`.

Severity breakdown

1 CRITICAL 1 HIGH 2 MEDIUM 2 LOW 2 INFO

Static sentinel pre-scan

Static sentinels matched 7 pattern(s) — 1 CRITICAL, 1 HIGH, 3 MEDIUM, 2 LOW.

CRITICAL SC05 · LZ-DVN-1 · line 290

lzReceive without trusted-sender check

lzReceive / _lzReceive must validate _origin.sender against peers[_origin.srcEid]; a missing check lets any chain forge messages.

```
function _lzReceive( Origin calldata _origin, bytes32 /*_guid*/, bytes calldata
_payload, address /*_executor*/, bytes calldata /*_extraData*/ ) internal override
whenNotPaused { if (_payl
```

HIGH SC08 · REE-1 · line 212

External call before state mutation (classic reentrancy)

Pattern call.value()() / .call{value:}() followed by state changes — checks-effects-interactions violated.

```
(bool ok, ) = payable(feeVault).call{value: magnetaServiceFee}("");
```

MEDIUM SC01 · FACT-1 · line 194

Factory create* function accepts paired arrays without length-match check

When `tokens[]` and `weights[]` (or any paired arrays) are passed together, the function must require `tokens.length == weights.length` — otherwise the trailing array is silently truncated and the pool ends up in an inconsistent state.

```
function createTokenAtomic( bytes calldata localParams, uint32[] calldata dstEids,
bytes[] calldata paramsPerChain, bytes calldata lzOptions, uint256 magnetaServiceFee
...)
```

MEDIUM SC01 · FACT-2 · line 10

Factory accepts address parameters without zero-address check

create*/deploy* functions that store `address` parameters (token, recipient, manager) without `require(\_x != address(0))` brick the resulting pool — token transfers revert, fees go to the zero address, etc.

```
function createForCreator — unchecked: creator
```

LOW SC02 · FACT-5 · line 10

Permissionless create* function with no rate-limit or registry guard

Open `createPool/createPair/deploy\*` is intentional for some launchpad designs, but it should at minimum emit a Creator-indexed event AND track the caller — otherwise attackers can grief gas/storage by spamming pools. Flag for review.

```
function createForCreator( address creator, string memory name, string memory symbol,
string memory tokenURI, uint256 totalSupply, bool revokeUpdate, bool revokeFreeze,
bool revokeMint ) external retu
```

LOW SC02 · FACT-5 · line 23

Permissionless create* function with no rate-limit or registry guard

Open `createPool/createPair/deploy\*` is intentional for some launchpad designs, but it should at minimum emit a Creator-indexed event AND track the caller — otherwise attackers can grief gas/storage by spamming pools. Flag for review.

```
function createForCreator( address creator, string memory name, string memory symbol,
string memory tokenURI, uint256 totalSupply, uint256 liquidityToBurn ) external
returns (address); } /// @title Cr
```

MEDIUM SC05 · RELAY-1 · line 290

Cross-chain message handler without nonce / replay tracking

Functions that consume a cross-chain payload (handle/receiveMessage/lzReceive) must dedupe with a nonce or processed-message map. The new 2026 attack class is replay-semantic divergence between chains.

```
function _lzReceive( Origin calldata _origin, bytes32 /*_guid*/, bytes calldata
_payload, address /*_executor*/, bytes calldata /*_extraData*/ ) internal override
whenNotPaused { if (_payl
```

Consolidated findings

CRITICAL CVSS 9.8 SC05 — Insecure Cross-Chain Messaging

Missing trusted-sender / peer validation in _lzReceive

_lzReceive decodes the incoming payload and calls _executeCreate (which deploys tokens with an arbitrary `creator`) without verifying that `\_origin.sender` matches the registered peer for `\_origin.srcEid`. Any contract on any LayerZero-connected chain can send a crafted message to this contract, causing it to deploy tokens with an attacker-controlled creator address. This is the exact attack vector exploited in the Kelp DAO incident (2026-04-19, \$292M). The OApp base class stores peers in `peers[srcEid]` and the check must be explicit in the override.

```
function _lzReceive (lines 290-295)
```

→ Add `require(\_origin.sender == peers[\_origin.srcEid], 'Untrusted sender');` as the first statement in `\_lzReceive`. Also enforce a minimum DVN count of 2 in the LZ configuration.

Confirmed by: static-sentinel, developer, historian, formalist

HIGH CVSS 7.5 SC08 — Reentrancy and Callback Vulnerabilities

External call to feeVault before state mutations and cross-chain sends (CEI violation)

In `createTokenAtomic`, `payable(feeVault).call{value: magnetaServiceFee}{'}` is executed at step 1 before local token creation (step 2) and cross-chain LZ sends (step 3). Although `nonReentrant` prevents same-contract reentry, a malicious or compromised `feeVault` contract can observe the pre-creation state, call back into other contracts, or cause the subsequent steps to fail after the fee has already been transferred. The checks-effects-interactions pattern requires all state changes to precede external calls.

```
createTokenAtomic, line 212
```

→ Move the feeVault transfer to the end of the function after all state changes and cross-chain sends, or use a pull-payment pattern where feeVault claims accumulated fees separately.

Confirmed by: static-sentinel, developer, historian, formalist

MEDIUM CVSS 5.3 SC05 — Insecure Cross-Chain Messaging

Cross-chain message replay relies solely on LayerZero GUID deduplication

The `\_lzReceive` handler does not maintain an independent processed-message map. While LayerZero V2 provides GUID-based deduplication at the protocol level, this creates a single point of trust in the LZ infrastructure. If the LZ endpoint or DVN configuration is ever misconfigured or upgraded, replay-semantic divergence between chains becomes possible. Defense-in-depth requires the application layer to track processed GUIDs. Note: this finding is secondary to the missing peer check and becomes the primary concern once that is fixed.

function `\_lzReceive` (lines 290–295)

→ Add a `mapping(bytes32 ⇒ bool) private \_processedGuids` and check/set it at the start of `\_lzReceive` using the `\_guid` parameter.

Confirmed by: static-sentinel, developer, historian, formalist

MEDIUM CVSS 4.3 SC01 — Improper Input Validation

Factory addresses can be set to zero without explicit intent guard

`setStandardFactory` and `setAutoLiquidityFactory` allow setting the factory to `address(0)` (the `NotAContract` check only fires for non-zero addresses with no code). The constructor also silently skips zero-address factories. While runtime calls will revert with `FactoryNotSet`, there is no event or explicit acknowledgement that the factory is being intentionally disabled, which can mask misconfiguration.

setStandardFactory (line ~155), setAutoLiquidityFactory (line ~161),
constructor (lines ~130–140)

→ Either disallow zero-address in setters (require non-zero) or emit a dedicated `FactoryDisabled` event when intentionally setting to zero, and document the intent clearly.

Confirmed by: historian, formalist

LOW CVSS 2.5 SC02 — Access Control Violations

Permissionless token creation with no rate-limit (interface-level observation)

The `createForCreator` functions in the factory interfaces are callable by this dispatcher without rate-limiting. While the Magneta service fee acts as an economic deterrent, a well-funded attacker could spam token creation to exhaust storage or grief the registry. The dispatcher itself does emit `TokenSpawned` events indexed by creator, which partially addresses tracking.

IMagnetaOFTStandardFactory.createForCreator (line 10),
IMagnetaOFTAutoLiquidityFactory.createForCreator (line 23)

→ Consider adding a per-block or per-address rate limit in the factory contracts, or a minimum service fee floor enforced on-chain in the dispatcher.

Confirmed by: static-sentinel, formalist

LOW CVSS 2.1 SC01 – Improper Input Validation

Zero-address creator not validated in cross-chain receive path

When `\_lzReceive` decodes the payload, the `creator` address is taken directly from the encoded bytes without a zero-address check. A malformed or malicious payload could set `creator = address(0)`, causing the factory to deploy a token with no owner. The source-chain path is safe (uses `msg.sender`), but the destination-chain path is not.

```
_lzReceive line 294: `(, address creator, bytes memory params) =  
abi.decode(...)`
```

→ Add `require(creator != address(0), 'Zero creator');` after decoding the payload in `\_lzReceive`.

Confirmed by: developer, formalist

INFO CVSS 0.0 SC01 – Improper Input Validation

FACT-1: Array length mismatch check is already implemented — false positive

The static sentinel flagged `createTokenAtomic` for missing paired-array length validation. The code at line 203 explicitly performs `if (paramsPerChain.length != n) revert ArrayLengthMismatch();`. This finding is a false positive and no action is required.

```
createTokenAtomic, line 203
```

→ No action required.

Confirmed by: developer

INFO CVSS 0.0 N/A

Upgradeability / timelock / oracle / ERC4626 invariants not applicable

The formalist agent generated findings for `\_authorizeUpgrade`, `TimelockController`, Chainlink oracle staleness, and ERC4626 `totalAssets`. None of these patterns exist in this contract. These findings are rejected as inapplicable.

N/A

→ No action required for this contract.

Historical precedents

Retrieved from the local bug-bounty corpus (~20 700 findings) by vulnerability-class match.

CRITICAL bridge ethereum 2022-03-29 \$624 000 000

Ronin Network — Bridge (2022-03-29)

<https://rekt.news/ronin-rekt/>

CRITICAL bridge ethereum 2021-08-11 \$611 000 000

Poly Network — Bridge, getting around modifier through cross-chain message (2021-08-11)

<https://rekt.news/polynetwork-rekt/>

CRITICAL bridge bsc 2022-01-28 \$80 000 000

Qubit Finance — Bridge address(0).safeTransferFrom() does not revert (2022-01-28)

<https://rekt.news/qubit-rekt/>

CRITICAL reentrancy ethereum 2022-04-30 \$80 000 000

Rari Capital/Fei Protocol — Flashloan Attack + Reentrancy (2022-04-30)

<https://certik.medium.com/fei-protocol-incident-analysis-8527440696cc>

CRITICAL reentrancy fantom 2021-12-18 \$30 000 000

Grim Finance — Flashloan & Reentrancy (2021-12-18)

<https://cointelegraph.com/news/defi-protocol-grim-finance-lost-30m-in-5x-reentrancy-hack>

CRITICAL reentrancy ethereum 2020-04-19 \$25 000 000

LendfMe — ERC777 Reentrancy (2020-04-19)

<https://peckshield.medium.com/uniswap-lendf-me-hacks-root-cause-and-loss-analysis-50f3263dcc09>

MEDIUM bm25 2023-04-01

rubicon — Anyone can create buddy for himself without calling

`BathHouseV2.createBathToken` function

<https://github.com/code-423n4/2023-04-rubicon-findings/issues/1172>

MEDIUM bm25 2024-08-01

phi — `RewardControl.withdrawFor` function can be called without `from` address' authorization, which can frontrun and DOS `from` address' `RewardControl.withdraw` function call and trigger `from` address' `receive` function's logics without its approval

<https://github.com/code-423n4/2024-08-phi-findings/issues/35>

MEDIUM bm25 2022-09-01

nouns-builder — Anyone can create a proposal without any voting power

<https://github.com/code-423n4/2022-09-nouns-builder-findings/issues/4>

HIGH bm25 2023-07-01

tapioca — `\_claimRewards` function lack a proper reentrancy guard, without properly handling the execution flow, it becomes susceptible to attacks.

<https://github.com/code-423n4/2023-07-tapioca-findings/issues/948>

HIGH bm25 2023-07-01

tapioca — user can withdraw collateral without check making complete without complete repayment

<https://github.com/code-423n4/2023-07-tapioca-findings/issues/892>

MEDIUM bm25 2024-01-01

curves — In the `FeeSplitter.sol#onBalanceChange` function, `token` is `push`ed to `userTokens` without performing a duplicate check.

<https://github.com/code-423n4/2024-01-curves-findings/issues/1201>

Generated by Sentinelleai on 2026-05-22T15:44:25.464Z. Audit ID: f196db8f-aa7b-4042-9212-089f8fe3900b.

This report combines deterministic regex+AST sentinels, optional Slither/Mythril static analysis, and a 6-model adversarial LLM panel consolidated by a Judge agent.